

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Trần Nguyễn Khải Luân
Nguyễn Bảo Duy
Trương Quốc Cường
Nguyễn Thanh Tiến

Nghiên cứu về khả năng học chính xác thuật toán của mạng nơ-ron

*Bài báo: Learning to Add, Multiply, and Execute Algorithmic Instructions
Exactly with Neural Networks*

Môn học: Nhập môn Học máy

TP. Hồ Chí Minh, Năm 2025

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Trần Nguyễn Khải Luân
Nguyễn Bảo Duy
Trương Quốc Cường
Nguyễn Thanh Tiến

Nghiên cứu về khả năng học chính xác thuật toán của mạng nơ-ron

*Bài báo: Learning to Add, Multiply, and Execute Algorithmic Instructions
Exactly with Neural Networks*

Môn học: Nhập môn Học máy

Giảng viên hướng dẫn: ThS. Lê Nhật Nam

TP. Hồ Chí Minh, Năm 2025

THÔNG TIN NHÓM

- **Nhóm:** 11
- **Danh sách thành viên:**
 1. Trần Nguyễn Khải Luân - 23127006
 2. Nguyễn Bảo Duy - 23127179
 3. Trương Quốc Cường - 23127333
 4. Nguyễn Thanh Tiến - 23127539

Mục lục

1	TỔNG QUAN	1
1.1	Giới thiệu	1
1.2	Các nghiên cứu liên quan	1
1.3	Động lực nghiên cứu	2
1.4	Câu hỏi nghiên cứu	2
1.5	Đóng góp của bài báo	3
2	CƠ SỞ LÝ THUYẾT	4
2.1	Ký hiệu và các khái niệm cơ bản	4
2.2	Miền liên tục và miền rời rạc: khác biệt về sai số	5
2.3	Đặc điểm thuật toán số học và ý tưởng template cục bộ	5
2.4	Tính đầy đủ Turing và lệnh SBN	5
2.5	Neural Tangent Kernel (NTK)	6
2.5.1	Khung chứng minh: ba mục tiêu chính	6
2.5.2	Thiết lập bài toán và ký hiệu động lực học	7
2.5.3	Mục tiêu 1: động lực học đầu ra trên tập huấn luyện	7
2.5.4	Mục tiêu 2: NTK regime và hiện tượng kernel đóng băng	8
2.5.5	Mục tiêu 3: giải ODE và công thức bộ dự đoán NTK	8
2.5.6	Mạng nơ-ron hai lớp trong chế độ NTK	9
2.5.7	Điều kiện để học chính xác và cơ chế làm tròn	10
3	PHƯƠNG PHÁP ĐỀ XUẤT	13
3.1	Kỹ thuật Template Matching	13
3.1.1	Nguyên lý phân rã thuật toán	13
3.1.2	Cơ chế thực thi lặp	14
3.2	Cấu trúc dữ liệu và Trục giao hóa	14
3.2.1	Mô hình “Bag of Rules”	14
3.2.2	Mã hóa Block-Partitioned	14
3.2.3	Trục giao hóa và Ma trận NTK	15
3.3	Định lý về bộ dự báo NTK	15
3.3.1	Định lý 5.1: Bảo toàn thông tin dấu	15
3.3.2	Ý nghĩa của Định lý	15
3.4	Kiểm soát tương quan không mong muốn	16
3.4.1	Assumption 5.1: Điều kiện về tương quan	16
3.4.2	Phân tích trường hợp xấu nhất	16
3.4.3	Kết quả cho các thuật toán cụ thể	16
3.5	Triển khai thuật toán cụ thể	17
3.5.1	Phép hoán vị	17

3.5.2	Phép cộng	17
3.5.3	Phép nhân	18
3.5.4	Chỉ thị SBN	18
3.6	Độ phức tạp tập hợp	18
3.6.1	Tại sao cần Ensemble?	19
3.6.2	Công thức tính số lượng mô hình tối thiểu	19
3.6.3	Độ phức tạp theo kích thước bài toán	19
4	THỰC NGHIỆM VÀ PHÂN TÍCH	20
4.1	Thiết lập thực nghiệm	20
4.1.1	Kiến trúc mạng	20
4.1.2	Tham số khởi tạo	20
4.1.3	Cấu hình huấn luyện	20
4.1.4	Đối tượng kiểm thử	21
4.2	Xác minh lý thuyết	21
4.2.1	Phương pháp xác minh	21
4.2.2	Kết quả trên phép cộng và nhân	21
4.2.3	So sánh phương sai và kỳ vọng	22
4.3	Thực nghiệm huấn luyện	22
4.3.1	Lựa chọn bài toán	22
4.3.2	Bài toán Hoán vị	23
4.3.3	Độ chính xác theo kích thước Ensemble	23
4.3.4	Phân tích kết quả	24
4.4	Tổng hợp và Đánh giá	24
5	KẾT LUẬN	25
5.1	Tổng kết nghiên cứu	25
5.2	Ý nghĩa của nghiên cứu	25
5.3	Hạn chế của phương pháp	26
5.4	Hướng phát triển tương lai	26
	Tài liệu tham khảo	27
A	Chứng minh toán học bổ trợ	31
A.1	Các kết quả đại số tuyến tính	31
A.1.1	Định lý Sherman-Morrison	31
A.2	Các kết quả xác suất	32
A.2.1	Bổ đề tập trung Gaussian	32
A.3	Chi tiết tính toán bộ dự báo NTK	32
A.3.1	Phân rã đầu ra NTK	32
A.3.2	Trọng số tín hiệu và nhiễu	33
A.3.3	Điều kiện để dấu đúng	33
A.4	Phân tích tiệm cận	33
A.4.1	Lemma 6.1: Bậc của phương sai và kỳ vọng	33
B	Mã nguồn cài đặt mô phỏng	35
B.1	Môi trường và thư viện	35
B.1.1	Thư viện Neural Tangents	35
B.1.2	Nền tảng JAX	35

B.2	Cấu trúc mã nguồn	35
B.2.1	Modules định nghĩa Template	36
B.2.2	Scripts mã hóa dữ liệu	36
B.2.3	Scripts tính toán Ensemble	36
B.3	Quy trình thực thi mô phỏng	36
B.3.1	Bước 1: Khởi tạo cấu trúc Block	36
B.3.2	Bước 2: Tạo tập dữ liệu huấn luyện	37
B.3.3	Bước 3: Tính toán ma trận NTK	37
B.3.4	Bước 4: Thực thi lặp và làm tròn	37
B.4	Kiểm chứng thực nghiệm	38
B.4.1	Trường hợp kiểm tra	38
B.4.2	Kết quả hội tụ	38

Danh sách bảng

Danh sách hình vẽ

Chương 1

TỔNG QUAN

1.1 Giới thiệu

Mạng nơ-ron đã chứng minh khả năng xấp xỉ vượt trội đối với các hàm liên tục và trơn, đặc biệt trong các bài toán nhận dạng mẫu, xử lý ngôn ngữ tự nhiên và thị giác máy tính. Tuy nhiên, khi đối mặt với các phép toán rời rạc như phép cộng nhị phân, phép nhân số học hay thực thi các chỉ thị thuật toán, mạng nơ-ron truyền thống thường gặp khó khăn đáng kể. Sự đối lập này phản ánh thách thức cơ bản: trong khi mạng nơ-ron được tối ưu hóa cho việc xấp xỉ hàm số trong không gian liên tục, các thuật toán thực tế yêu cầu sự chính xác tuyệt đối ở cấp độ bit.

Nghiên cứu đã đề xuất một hướng tiếp cận đột phá để giải quyết vấn đề này thông qua khung lý thuyết Neural Tangent Kernel (NTK). Bằng cách phân tích mạng nơ-ron hai lớp trong giới hạn chiều rộng vô hạn, các tác giả chứng minh rằng mạng nơ-ron có thể được huấn luyện để thực thi chính xác các chỉ thị thuật toán được mã hóa ở dạng nhị phân. Đây là kết quả có ý nghĩa lý thuyết quan trọng, mở ra khả năng sử dụng mạng nơ-ron như một công cụ tính toán phổ quát thay vì chỉ giới hạn ở vai trò xấp xỉ hàm số.

Mục tiêu cốt lõi của nghiên cứu là chứng minh một cách chặt chẽ về mặt toán học rằng mạng nơ-ron, khi được huấn luyện thông qua gradient descent với cơ chế template matching, có khả năng học và thực thi chính xác các chỉ thị thuật toán mà không cần xấp xỉ.

1.2 Các nghiên cứu liên quan

Khả năng tính toán của mạng nơ-ron đã được nghiên cứu rộng rãi trong nhiều thập kỷ qua. Các nghiên cứu tiên phong về mạng nơ-ron hồi quy (RNN) đã chứng minh rằng RNN có thể mô phỏng máy Turing về mặt lý thuyết, nghĩa là chúng có khả năng biểu diễn bất kỳ hàm tính toán được nào. Gần đây hơn, kiến trúc Transformer cũng được chứng minh có khả năng biểu diễn tương đương, củng cố thêm quan điểm về tính phổ quát của mạng nơ-ron.

Tuy nhiên, điều quan trọng cần phân biệt là tính khả thi về mặt biểu diễn hoàn toàn khác với khả năng học được thông qua gradient descent. Một mạng nơ-ron có thể có khả năng biểu diễn một hàm phức tạp, nhưng điều đó không đảm bảo rằng thuật toán tối ưu hóa như gradient descent có thể tìm được tập tham số phù hợp để thực hiện hàm đó từ dữ liệu huấn luyện hữu hạn. Đây chính là khoảng cách then chốt mà các nghiên cứu trước đó chưa giải quyết triệt để.

Nhiều kiến trúc chuyên biệt đã được đề xuất để thu hẹp khoảng cách này. Neural GPU mở rộng khả năng của mạng tích chập để xử lý các phép toán số học song song. Neural Arithmetic Logic Unit (NALU) được thiết kế đặc biệt để học các phép toán số học như cộng, trừ, nhân và chia. Mặc dù các kiến trúc này đạt được kết quả thực nghiệm tốt trong một số trường hợp, chúng thiếu nền tảng lý thuyết vững chắc về việc tại sao và khi nào quá trình học hội tụ đến giải pháp chính xác.

Nghiên cứu đã cho thấy sự khác biệt cơ bản ở chỗ chứng minh lý thuyết chặt chẽ dựa trên khung NTK, đảm bảo rằng mạng nơ-ron không chỉ có khả năng biểu diễn mà thực sự có thể học được các phép toán chính xác thông qua gradient descent với lượng dữ liệu huấn luyện có thể kiểm soát được.

1.3 Động lực nghiên cứu

Một trong những thách thức cốt lõi khi huấn luyện mạng nơ-ron thực hiện các phép toán rời rạc là sự khó khăn trong việc hội tụ gradient descent đối với các hàm không liên tục. Gradient descent hoạt động dựa trên việc tính toán đạo hàm của hàm mất mát theo các tham số mạng, nhưng đối với các hàm bước nhảy hay các phép toán nhị phân, đạo hàm hầu như bằng không hoặc không xác định tại hầu hết các điểm. Điều này dẫn đến hiện tượng gradient biến mất hoặc gradient không cung cấp thông tin hữu ích cho việc cập nhật tham số.

Các bộ dữ liệu tiêu chuẩn cho bài toán số học thường không được thiết kế để giải quyết vấn đề này một cách có hệ thống. Hệ quả là mạng nơ-ron được huấn luyện trên các bộ dữ liệu như vậy có thể đạt độ chính xác cao trên tập huấn luyện nhưng thất bại khi khái quát hóa sang các ví dụ mới, đặc biệt khi kích thước đầu vào tăng lên.

Động lực chính của nghiên cứu này xuất phát từ nhu cầu tìm kiếm một cơ chế huấn luyện có thể đảm bảo mạng nơ-ron đạt được độ chính xác tuyệt đối thay vì chỉ xấp xỉ. Thay vì cố gắng xấp xỉ một hàm rời rạc bằng một hàm liên tục, nghiên cứu này đề xuất mã hóa dữ liệu và thuật toán theo cách cho phép mạng nơ-ron nhận dạng và áp dụng các quy tắc cấp độ bit một cách chính xác thông qua cơ chế template matching. Cách tiếp cận này biến bài toán từ xấp xỉ hàm thành bài toán nhận dạng mẫu, nơi mạng nơ-ron có thể phát huy thế mạnh vốn có.

1.4 Câu hỏi nghiên cứu

Nghiên cứu đặt ra một câu hỏi tổng quát, gồm **ba phần** tương ứng với (i) khía cạnh **lý thuyết**, (ii) khía cạnh **hiệu quả dữ liệu**, và (iii) khía cạnh **kết nối với thực nghiệm**:

*(1) Về mặt lý thuyết, liệu có thể chứng minh rằng một mạng nơ-ron học được cơ chế template matching để thực thi **chính xác** các chỉ thị thuật toán, dưới các giả định như NTK và mô hình nhiễu; (2) lượng dữ liệu huấn luyện tối thiểu cần thiết (độ phức tạp mẫu) là bao nhiêu; và (3) trong thực tế khi mạng có độ rộng hữu hạn và dữ liệu hữu hạn, các bảo đảm/dự đoán từ lý thuyết được phản ánh như thế nào?*

Từ đó, báo cáo cụ thể hóa câu hỏi nghiên cứu thành **ba phần**:

- **Phần 1 (Lý thuyết/NTK).** Trong khuôn khổ NTK ở giới hạn chiều rộng vô hạn, liệu gradient descent có thể hội tụ đến nghiệm thực thi **đúng** thuật toán (ở mức bit) hay không?

- **Phần 2 (Thực nghiệm/mạng hữu hạn).** Khi rời khỏi giả định chiều rộng vô hạn (mạng hữu hạn và dữ liệu hữu hạn), mô hình còn có thể học và khái quát hóa việc thực thi chỉ thị thuật toán ở mức độ nào? Mức suy giảm so với dự đoán/bảo đảm lý thuyết thể hiện ra sao theo độ dài bit, theo seed/khởi tạo, và theo quy mô dữ liệu?
- **Phần 3 (Độ phức tạp mẫu).** Lượng dữ liệu huấn luyện tối thiểu phụ thuộc như thế nào vào kích thước đầu vào?

1.5 Đóng góp của bài báo

Nghiên cứu của De Luca và cộng sự đã mang lại những đóng góp quan trọng về mặt lý thuyết và phương pháp luận:

- **Biểu diễn thuật toán dưới dạng template:** Mỗi quy tắc cấp độ bit của thuật toán được mã hóa thành một vector mẫu riêng biệt, cho phép cô lập các quy tắc logic và tạo ra cấu trúc dữ liệu phù hợp cho việc áp dụng cơ chế template matching.
- **Kiểm soát tương quan trong chế độ NTK:** Bằng cách điều chỉnh sự tương quan giữa các mẫu trong chế độ Neural Tangent Kernel, nghiên cứu đảm bảo rằng dự đoán của mô hình đồng nhất với việc thực thi thuật toán mục tiêu.
- **Phương pháp Ensemble:** Chứng minh rằng một tập hợp (ensemble) đủ lớn các mạng nơ-ron hai lớp trong giới hạn chiều rộng vô hạn có thể đạt độ chính xác tuyệt đối với xác suất cao cho các tác vụ như phép hoán vị, phép cộng, phép nhân và chỉ thị SBN.
- **Hiệu quả về dữ liệu:** Phương pháp chỉ cần số lượng ví dụ huấn luyện ở quy mô logarithmic thay vì tuyến tính hoặc đa thức, cho phép huấn luyện hiệu quả ngay cả với các bài toán có kích thước đầu vào lớn.
- **Tính phổ quát:** Vì chỉ thị SBN là đầy đủ Turing, việc chứng minh mạng nơ-ron có thể học chính xác SBN đồng nghĩa với việc mạng nơ-ron có thể, về nguyên tắc, học thực thi bất kỳ thuật toán nào.
- **Góc nhìn giáo dục (của báo cáo này):** Báo cáo tổng hợp và trình bày lại các kết quả từ nghiên cứu gốc, giúp người đọc tiếp cận các khái niệm phức tạp như NTK, giới hạn chiều rộng vô hạn, và template matching một cách có hệ thống.

Các đóng góp này không chỉ có giá trị về mặt lý thuyết thuần túy mà còn mở ra hướng phát triển mới cho việc thiết kế và huấn luyện mạng nơ-ron trong các ứng dụng yêu cầu độ chính xác tuyệt đối, như xác minh phần cứng, mã hóa và giải mã, và các hệ thống nhúng quan trọng.

Chương 2

CƠ SỞ LÝ THUYẾT

2.1 Ký hiệu và các khái niệm cơ bản

Trước khi đi vào phân tích chi tiết khung lý thuyết Neural Tangent Kernel (NTK), chúng ta cần thiết lập các ký hiệu toán học cơ bản được sử dụng xuyên suốt báo cáo này.

Quy ước ký hiệu: Ta dùng chữ **in đậm** để chỉ vector (ví dụ $\mathbf{x}$). Hai ký hiệu $\mathbf{1}$ và $\mathbf{0}$ lần lượt là vector toàn 1 và toàn 0 với độ dài phù hợp theo ngữ cảnh.

Vector và ma trận: Một vector cột n chiều được ký hiệu là $\mathbf{x} \in \mathbb{R}^n$, trong đó phần tử thứ i được ký hiệu là x_i hoặc $[\mathbf{x}]_i$. Ma trận đơn vị kích thước $n \times n$ được ký hiệu là I_n . Tích vô hướng giữa hai vector $\mathbf{x}$ và $\mathbf{y}$ được ký hiệu là $\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y} = \sum_{i=1}^n x_i y_i$.

Chuẩn Euclidean: Chuẩn Euclidean (hay chuẩn L^2) của một vector $\mathbf{x}$ được định nghĩa là:

$$\|\mathbf{x}\| = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle} \quad (2.1)$$

Tập hợp chỉ số: Ký hiệu $[n]$ biểu thị tập hợp các số nguyên dương từ 1 đến n :

$$[n] = \{1, 2, \dots, n\} \quad (2.2)$$

Góc giữa hai vector: Với hai vector khác không $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^n$, ta ký hiệu góc giữa chúng là

$$\theta(\mathbf{x}, \mathbf{x}') = \arccos \left(\frac{\langle \mathbf{x}, \mathbf{x}' \rangle}{\|\mathbf{x}\| \|\mathbf{x}'\|} \right) \in [0, \pi]. \quad (2.3)$$

Bài toán hồi quy và hàm loss: Xét một mạng nơ-ron với tham số θ thực hiện ánh xạ đầu vào $\mathbf{x}$ sang đầu ra $F(\mathbf{x}; \theta)$. Cho tập dữ liệu huấn luyện $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$ với N mẫu, mục tiêu huấn luyện là tối thiểu hóa hàm loss Mean Squared Error (MSE):

$$\mathcal{L}(\theta) = \frac{1}{2} \|F(\mathbf{x}; \theta) - \mathbf{y}\|^2 = \frac{1}{2} \sum_{i=1}^N \|F(\mathbf{x}_i; \theta) - \mathbf{y}_i\|^2 \quad (2.4)$$

Trong đó $\mathbf{y}$ biểu thị vector chứa tất cả các nhãn đầu ra mục tiêu. Hàm loss MSE có đạo hàm đơn giản và là lựa chọn tiêu chuẩn cho các bài toán hồi quy, đồng thời cũng là nền tảng cho việc phân tích lý thuyết trong khung NTK.

2.2 Miền liên tục và miền rời rạc: khác biệt về sai số

Một điểm xuất phát quan trọng của bài báo là sự khác biệt giữa hai kiểu bài toán mà học sâu thường gặp:

- **Bài toán liên tục:** đầu vào/đầu ra mang tính trơn. Sai số nhỏ thường không làm thay đổi ý nghĩa kết quả (ví dụ $0.99 \approx 1.00$).
- **Bài toán rời rạc:** đầu vào/đầu ra ở mức bit; chỉ cần sai một bit là kết quả có thể sai hoàn toàn, không còn khái niệm “gần đúng”.

Do đó, việc tối ưu trong không gian liên tục bằng hạ gradient không đương nhiên dẫn đến nghiệm thực thi **đúng tuyệt đối** một quy tắc rời rạc. Nhận xét này tạo động lực cho việc đưa ra một khung phân tích trong đó có thể phát biểu và chứng minh tính học được cho các hàm rời rạc [1].

2.3 Đặc điểm thuật toán số học và ý tưởng template cục bộ

Các thuật toán số học cổ điển như cộng/nhân nhị phân có cấu trúc phù hợp để phân rã thành các quy tắc cục bộ học được từ dữ liệu. Ba đặc tính thường được sử dụng là:

- **Cục bộ:** mỗi bước chỉ phụ thuộc vào một phần rất nhỏ của trạng thái (ví dụ: hai bit và bit nhớ), cho phép mô tả dưới dạng quy tắc cục bộ.
- **Lặp lại:** cùng một quy tắc được áp dụng lặp đi lặp lại theo vị trí bit, tạo điều kiện cho việc lặp khi thực thi chuỗi dài.
- **Gần độc lập theo vị trí:** quy tắc tại vị trí i chủ yếu phụ thuộc vào thông tin lân cận, giúp tách thuật toán thành các bước nhỏ có thể học và phân tích độc lập.

Trên nền tảng đó, bài báo mô hình hóa việc học thuật toán như việc học đúng các ánh xạ cục bộ $X \rightarrow Y$ trên các khối trạng thái, sau đó **lặp** các ánh xạ đã học để thực thi thuật toán hoàn chỉnh [1].

2.4 Tính đầy đủ Turing và lệnh SBN

Một hệ thống được gọi là **đầy đủ Turing** nếu nó có thể mô phỏng mọi thuật toán khả tính (tương đương một máy Turing) [27, 24]. Ở mức trực giác, để đạt đầy đủ Turing, một tập lệnh/mô hình tính toán cần có (ít nhất) ba năng lực cốt lõi:

1. **Thao tác bộ nhớ:** có khả năng cập nhật trạng thái/bộ nhớ (đọc–ghi–biến đổi các biến).
2. **Logic điều kiện:** có khả năng kiểm tra một điều kiện trên trạng thái hiện tại để quyết định hành động tiếp theo.
3. **Điều khiển luồng/vòng lặp:** có cơ chế nhảy/vòng lặp để thực thi chuỗi thao tác với số bước không bị chặn trước.

Lệnh **SBN** có dạng “*trừ và nếu kết quả âm thì nhảy*”, nên thỏa cả cập nhật trạng thái, điều kiện, và điều khiển luồng; do đó có thể đạt đầy đủ Turing khi được lặp lại/ghép chuỗi phù hợp [25, 24].

Tác giả đưa SBN vào để nhấn mạnh rằng khung template và phân tích NTK không chỉ nhắm tới các phép toán số học cụ thể, mà còn đóng vai trò **nền tảng lý thuyết** hướng tới việc học và thực thi **mọi thuật toán** [1].

2.5 Neural Tangent Kernel (NTK)

Neural Tangent Kernel (NTK) là một công cụ lý thuyết mạnh mẽ cho phép phân tích động lực học huấn luyện của mạng nơ-ron [2]. Ý tưởng cốt lõi là mô tả sự thay đổi của đầu ra mạng theo thời gian huấn luyện thông qua một kernel gần như cố định trong chế độ chiều rộng lớn (chế độ huấn luyện lười) [2, 5].

2.5.1 Khung chứng minh: ba mục tiêu chính

Để chứng minh lý thuyết của NTK, ta tập trung vào ba mục tiêu chính:

1. **Động lực học của đầu ra trên tập huấn luyện:** thiết lập phương trình vi phân cho vector đầu ra $\mathbf{f}(t)$ dưới gradient flow,

$$\frac{d}{dt}\mathbf{f}(t) = -K(\theta(t))(\mathbf{f}(t) - \mathbf{y}).$$

trong đó $K(\theta) = J(\theta)J(\theta)^\top$ là NTK thực nghiệm.

2. **Chế độ NTK / huấn luyện lười:** khi chiều rộng $m \rightarrow \infty$, kernel gần như *đóng băng theo thời gian*, và hội tụ về một kernel giới hạn xác định Θ [2, 5]:

$$K(\theta(t)) \approx \Theta(\mathcal{X}, \mathcal{X}) \quad \text{trong } t \in [0, T].$$

3. **Giải ODE và liên hệ kernel regression:** thay $K(\theta(t))$ bởi Θ (xấp xỉ hằng theo t), ta giải được nghiệm giới hạn và nhận được công thức bộ dự đoán NTK tương đương kernel regression [2, 9].

Ma trận Jacobian: Xét mạng nơ-ron với đầu ra $F(\mathbf{x}; \theta) \in \mathbb{R}^k$ phụ thuộc vào vector tham số $\theta \in \mathbb{R}^p$. Ma trận Jacobian của đầu ra theo tham số được định nghĩa là:

$$J(\theta) = \frac{\partial F(\mathbf{x}; \theta)}{\partial \theta} \in \mathbb{R}^{Nk \times p}$$

trong đó N là số lượng mẫu dữ liệu, k là chiều của đầu ra, và p là số lượng tham số. Mỗi hàng của ma trận Jacobian biểu thị gradient của một thành phần đầu ra theo toàn bộ tham số.

Định nghĩa NTK thực nghiệm: Neural Tangent Kernel thực nghiệm được định nghĩa thông qua tích của ma trận Jacobian với chuyển vị của nó:

$$K(\theta) = J(\theta)J(\theta)^\top \in \mathbb{R}^{Nk \times Nk}$$

Phần tử (i, j) của ma trận kernel này đo lường mức độ tương quan giữa gradient của đầu ra thứ i và đầu ra thứ j trong không gian tham số. Cụ thể hơn, với hai điểm dữ liệu $\mathbf{x}$ và $\mathbf{x}'$, giá trị kernel được tính như sau:

$$K(\mathbf{x}, \mathbf{x}'; \theta) = \left\langle \frac{\partial F(\mathbf{x}; \theta)}{\partial \theta}, \frac{\partial F(\mathbf{x}'; \theta)}{\partial \theta} \right\rangle$$

Ý nghĩa vật lý của NTK: NTK cho phép hiểu rõ cách mạng nơ-ron “học” từ dữ liệu. Khi huấn luyện bằng gradient descent, sự thay đổi của đầu ra mạng tại một điểm $\mathbf{x}$ không chỉ phụ thuộc vào gradient tại điểm đó mà còn phụ thuộc vào gradient tại tất cả các điểm dữ liệu khác thông qua kernel. Đây chính là cơ chế “học chuyển giao” (transfer learning) nội tại trong mạng nơ-ron: thông tin học được từ một mẫu có thể ảnh hưởng đến dự đoán tại các mẫu khác.

2.5.2 Thiết lập bài toán và ký hiệu động lực học

Xét tập huấn luyện $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$. Đặt $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ và gom vector đầu ra trên tập train:

$$\mathbf{f}(\theta) = \begin{bmatrix} F(\mathbf{x}_1; \theta) \\ \vdots \\ F(\mathbf{x}_N; \theta) \end{bmatrix} \in \mathbb{R}^{Nk}, \quad \mathbf{y} = \begin{bmatrix} \mathbf{y}_1 \\ \vdots \\ \mathbf{y}_N \end{bmatrix}.$$

Ta dùng hàm mất mát MSE $\mathcal{L}(\theta) = \frac{1}{2} \|\mathbf{f}(\theta) - \mathbf{y}\|^2$ và xét **gradient flow**:

$$\frac{d\theta(t)}{dt} = -\nabla_{\theta} \mathcal{L}(\theta(t)).$$

2.5.3 Mục tiêu 1: động lực học đầu ra trên tập huấn luyện

Một trong những đóng góp quan trọng nhất của lý thuyết NTK là khám phá hành vi của mạng nơ-ron khi chiều rộng lớp ẩn tiến tới vô hạn [2]. Trong giới hạn này, việc huấn luyện mạng nơ-ron trở nên đơn giản hóa đáng kể và có thể được phân tích chính xác.

Từ định nghĩa $\mathcal{L}(\theta) = \frac{1}{2} \|\mathbf{f}(\theta) - \mathbf{y}\|^2$ ta có

$$\nabla_{\theta} \mathcal{L}(\theta) = J(\theta)^{\top} (\mathbf{f}(\theta) - \mathbf{y}).$$

Do đó gradient flow cho tham số là

$$\frac{d\theta(t)}{dt} = -J(\theta(t))^{\top} (\mathbf{f}(t) - \mathbf{y}).$$

Áp dụng quy tắc dây chuyền cho $\mathbf{f}(t) = \mathbf{f}(\theta(t))$:

$$\frac{d\mathbf{f}(t)}{dt} = J(\theta(t)) \frac{d\theta(t)}{dt} = -J(\theta(t)) J(\theta(t))^{\top} (\mathbf{f}(t) - \mathbf{y}).$$

Đặt $K(\theta) = J(\theta) J(\theta)^{\top}$, ta thu được phương trình mục tiêu:

$$\frac{d\mathbf{f}(t)}{dt} = -K(\theta(t)) (\mathbf{f}(t) - \mathbf{y}).$$

2.5.4 Mục tiêu 2: NTK regime và hiện tượng kernel đóng băng

Vấn đề còn lại là $K(\theta(t))$ vẫn phụ thuộc vào thời gian thông qua $\theta(t)$. Lý thuyết NTK cho thấy rằng khi chiều rộng $m \rightarrow \infty$, ta có (trong một khoảng thời gian hữu hạn $[0, T]$) [2, 5]:

$$\sup_{t \in [0, T]} \|K_m(\theta(t)) - K_m(\theta(0))\| \rightarrow 0,$$

và đồng thời $K_m(\theta(0))$ hội tụ về một kernel giới hạn xác định $\Theta(\mathcal{X}, \mathcal{X})$.

Bước 2.1: hội tụ của NTK tại khởi tạo

Với mạng hai lớp độ rộng m , NTK thực nghiệm tại khởi tạo có thể viết như tổng theo các neuron ẩn:

$$K_{m,ij}(0) = \sum_{r=1}^m \langle g_{ir}(0), g_{jr}(0) \rangle,$$

trong đó $g_{ir}(0) = \nabla_{\theta_r} F(\mathbf{x}_i; \theta(0))$ là gradient của đầu ra tại điểm $\mathbf{x}_i$ theo tham số của neuron ẩn thứ r . Với tham số khởi tạo i.i.d., các hạng tử theo r là i.i.d. (và có kỳ vọng hữu hạn), do đó theo luật số lớn, khi $m \rightarrow \infty$ ta thu được

$$K_m(\theta(0)) \rightarrow \Theta(\mathcal{X}, \mathcal{X}).$$

Bước 2.2: lazy training và kernel gần như không đổi

Trực giác chính của lazy training là: với chuẩn hóa NTK, mỗi neuron đóng góp cỡ nhỏ vào gradient, nên từng tham số thay đổi rất ít trong thời gian hữu hạn. Khi kết hợp với tính Lipschitz (cục bộ) của gradient theo tham số, ta suy ra $g_{ir}(t)$ thay đổi rất nhỏ, và vì K là tổng các tích trong $\langle g_{ir}(t), g_{jr}(t) \rangle$, tổng thể $K_m(\theta(t))$ gần như không đổi theo t .

2.5.5 Mục tiêu 3: giải ODE và công thức bộ dự đoán NTK

Khi chiều rộng lớp ẩn $m \rightarrow \infty$, mạng nơ-ron hoạt động trong chế độ được gọi là *lazy training*. Trong chế độ này, các tham số của mạng thay đổi cực nhỏ quanh điểm khởi tạo trong suốt quá trình huấn luyện:

$$\|\theta(t) - \theta(0)\| \approx O\left(\frac{1}{\sqrt{m}}\right) \rightarrow 0 \quad \text{khi } m \rightarrow \infty$$

Điều này có vẻ nghịch lý: làm sao mạng có thể học được nếu tham số hầu như không thay đổi? Câu trả lời nằm ở việc với số lượng tham số cực lớn, ngay cả những thay đổi nhỏ trong mỗi tham số cũng đủ để tạo ra thay đổi đáng kể trong đầu ra. Đây chính là sức mạnh của việc có nhiều chiều trong không gian tham số.

Sự ổn định của kernel (hiện tượng đóng băng)

Một hệ quả quan trọng của chế độ huấn luyện lười là NTK trở nên ổn định (“đóng băng”) trong suốt quá trình huấn luyện. Cụ thể, khi $m \rightarrow \infty$:

$$K_m(\theta(t)) \rightarrow \Theta(\mathbf{x}, \mathbf{x}') \quad \text{với mọi } t \geq 0$$

trong đó $\Theta(\mathbf{x}, \mathbf{x}')$ là NTK ở giới hạn chiều rộng vô hạn, một kernel xác định và cố định chỉ phụ thuộc vào cặp đầu vào $(\mathbf{x}, \mathbf{x}')$ và kiến trúc mạng, không phụ thuộc vào quá trình huấn luyện. Sự ổn định này là chìa khóa cho phép phân tích chính xác động lực học huấn luyện.

Phương trình vi phân cho đầu ra mạng

Trong NTK regime, ta thay $K(\theta(t))$ bởi $\Theta(\mathcal{X}, \mathcal{X})$ (hằng theo t) để được ODE tuyến tính:

$$\frac{d\mathbf{f}(t)}{dt} = -\Theta(\mathcal{X}, \mathcal{X}) (\mathbf{f}(t) - \mathbf{y}).$$

Đặt $\mathbf{u}(t) = \mathbf{f}(t) - \mathbf{y}$, ta có $\frac{d\mathbf{u}(t)}{dt} = -\Theta(\mathcal{X}, \mathcal{X})\mathbf{u}(t)$ và nghiệm tường minh:

$$\mathbf{f}(t) = \mathbf{y} + e^{-\Theta(\mathcal{X}, \mathcal{X})t}(\mathbf{f}(0) - \mathbf{y}).$$

Nếu $\Theta(\mathcal{X}, \mathcal{X})$ xác định dương, khi $t \rightarrow \infty$ ta có $\mathbf{f}(t) \rightarrow \mathbf{y}$ (fit đúng trên tập train).

Công thức tại điểm test và tương đương kernel regression

Khi xét một điểm test $\mathbf{x}$, động lực học của $F(\mathbf{x}; \theta(t))$ cũng có thể biểu diễn qua kernel giới hạn. Bỏ qua các chi tiết kỹ thuật, nghiệm giới hạn của bộ dự đoán NTK có thể viết dưới dạng:

$$F_\infty(\mathbf{x}) = F(\mathbf{x}; \theta(0)) - \Theta(\mathbf{x}, \mathcal{X}) \Theta(\mathcal{X}, \mathcal{X})^{-1} (\mathbf{f}(0) - \mathbf{y}).$$

Trong nhiều trường hợp (hoặc sau khi lấy kỳ vọng theo khởi tạo), ta có thể xem $F(\mathbf{x}; \theta(0)) \approx 0$ và khi đó công thức rút gọn thành

$$F_\infty(\mathbf{x}) = \Theta(\mathbf{x}, \mathcal{X}) \Theta(\mathcal{X}, \mathcal{X})^{-1} \mathbf{y}.$$

Đây chính là nghiệm của hồi quy kernel với kernel Θ [2, 9].

trong đó:

- $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ là tập các điểm huấn luyện
- $\Theta(\mathbf{x}^*, \mathcal{X}) \in \mathbb{R}^{1 \times N}$ là vector kernel giữa điểm test và các điểm huấn luyện
- $\Theta(\mathcal{X}, \mathcal{X}) \in \mathbb{R}^{N \times N}$ là ma trận Gram trên tập huấn luyện

Công thức này chính là nghiệm của bài toán kernel regression tiêu chuẩn [9]. Điều này cho thấy mạng nơ-ron sâu, trong giới hạn chiều rộng vô hạn, hoạt động như một phương pháp kernel cổ điển với một kernel đặc biệt được xác định bởi kiến trúc mạng [2].

2.5.6 Mạng nơ-ron hai lớp trong chế độ NTK

Nghiên cứu của Back de Luca và cộng sự tập trung vào mạng nơ-ron hai lớp với kiến trúc đơn giản nhưng đủ mạnh để phân tích lý thuyết [1].

Cấu trúc mạng

Mạng nơ-ron hai lớp được sử dụng có cấu trúc như sau:

- **Lớp đầu vào:** Nhận vector $\mathbf{x} \in \mathbb{R}^d$
- **Lớp ẩn:** Gồm m nơ-ron với hàm kích hoạt ReLU, không sử dụng bias
- **Lớp đầu ra:** Ánh xạ tuyến tính từ m nơ-ron ẩn sang k đầu ra

Công thức toán học của mạng được viết là:

$$F(\mathbf{x}) = W^2 \cdot \text{ReLU}(W^1 \mathbf{x})$$

trong đó:

- $W^1 \in \mathbb{R}^{m \times d}$ là ma trận trọng số lớp thứ nhất
- $W^2 \in \mathbb{R}^{k \times m}$ là ma trận trọng số lớp thứ hai
- $\text{ReLU}(z) = \max(0, z)$ là hàm kích hoạt Rectified Linear Unit, áp dụng element-wise

Việc không sử dụng bias đơn giản hóa phân tích lý thuyết mà không làm mất tính tổng quát về mặt biểu diễn, vì bias có thể được mô phỏng bằng cách thêm một chiều hằng số vào đầu vào.

Chuẩn hóa tham số theo NTK

Để đảm bảo tính hội tụ trong giới hạn chiều rộng vô hạn, các trọng số được khởi tạo theo chuẩn tham số hóa NTK [2, 1]. Cụ thể:

- Các phần tử của W^1 được khởi tạo i.i.d. từ phân phối $\mathcal{N}(0, 1)$
- Các phần tử của W^2 được khởi tạo i.i.d. từ phân phối $\mathcal{N}(0, 1/m)$

Hệ số $1/m$ trong phương sai của W^2 là then chốt. Nó đảm bảo rằng đầu ra của mạng có phương sai $O(1)$ khi $m \rightarrow \infty$:

$$\text{Var}[F(\mathbf{x})] = \text{Var} \left[\sum_{j=1}^m W_{ij}^2 \cdot \text{ReLU}(W_j^1 \mathbf{x}) \right] \approx m \cdot \frac{1}{m} \cdot O(1) = O(1)$$

Nếu không có hệ số này, đầu ra sẽ có phương sai tăng theo m , làm mất tính ổn định khi $m \rightarrow \infty$.

2.5.7 Điều kiện để học chính xác và cơ chế làm tròn

Để mạng nơ-ron có thể học thực thi chính xác các chỉ thị thuật toán, cần có các điều kiện đảm bảo rằng thông tin tín hiệu không bị nhiễu làm suy giảm. Phần này trình bày các điều kiện then chốt được đề xuất trong nghiên cứu.

Assumption 5.1: Kiểm soát tương quan không mong muốn

Xét một bước tính toán của thuật toán với đầu vào được biểu diễn dưới dạng template. Đầu ra của bộ dự đoán NTK có thể được phân tích thành hai thành phần:

$$\hat{y} = w^1 \cdot (\text{tín hiệu chính xác}) + w^0 \cdot (\text{tổng các tương quan không mong muốn})$$

trong đó:

- w^1 là *trọng số tín hiệu*: Đóng góp từ các template khớp chính xác với quy tắc cần áp dụng
- w^0 là *trọng số nhiễu*: Đóng góp từ các template không liên quan nhưng có tương quan khác không với đầu vào

Định nghĩa tương quan không mong muốn: Một tương quan được gọi là “không mong muốn” (unwanted correlation) khi một template trong tập huấn luyện có tích vô hướng khác không với đầu vào hiện tại, mặc dù template đó không đại diện cho quy tắc cần áp dụng. Gọi C là số lượng các tương quan không mong muốn như vậy.

Điều kiện đủ để học chính xác: Để đầu ra $\hat{y}$ có dấu đúng (từ đó có thể làm tròn về giá trị chính xác), cần đảm bảo rằng thành phần tín hiệu lấn át thành phần nhiễu:

$$|w^1| > C \cdot |w^0|$$

hay tương đương:

$$C < \frac{|w^1|}{|w^0|}$$

Điều kiện này có ý nghĩa trực quan: số lượng tương quan không mong muốn phải đủ nhỏ so với tỷ lệ giữa trọng số tín hiệu và trọng số nhiễu. Trong thiết kế template của nghiên cứu này, các tác giả đã chứng minh rằng với cách mã hóa phù hợp, tỷ lệ $|w^1|/|w^0|$ tăng theo kích thước bài toán, trong khi C chỉ tăng theo logarithm, đảm bảo điều kiện trên được thỏa mãn [1].

Vai trò của hàm làm tròn

Một thành phần quan trọng trong khung lý thuyết là việc áp dụng hàm làm tròn sau mỗi bước dự đoán. Cụ thể:

$$y_{\text{output}} = \text{round}(\hat{y}) = \begin{cases} +1 & \text{nếu } \hat{y} > 0 \\ -1 & \text{nếu } \hat{y} < 0 \end{cases}$$

Hàm làm tròn đóng vai trò then chốt trong việc:

1. **Loại bỏ nhiễu:** Miễn là đầu ra thô $\hat{y}$ có dấu đúng, giá trị chính xác được khôi phục hoàn toàn sau khi làm tròn. Điều này cho phép chấp nhận một lượng nhiễu nhất định mà không ảnh hưởng đến kết quả cuối cùng.
2. **Ngăn chặn tích lũy sai số:** Trong các thuật toán đa bước, nếu không có làm tròn, sai số nhỏ ở các bước trước sẽ tích lũy và phóng đại qua các bước sau. Làm tròn tại mỗi bước đảm bảo rằng đầu ra luôn chính xác tuyệt đối, ngăn chặn hiện tượng này.

3. **Chuyển đổi xấp xỉ thành chính xác:** Trong khi bộ dự đoán NTK về bản chất là một bộ xấp xỉ liên tục, việc kết hợp với làm tròn biến nó thành một hệ thống thực thi chính xác các phép toán rời rạc.

Kết hợp Assumption 5.1 và cơ chế làm tròn, nghiên cứu chứng minh rằng mạng nơ-ron trong giới hạn infinite-width có khả năng học và thực thi chính xác các chỉ thị thuật toán, mở đường cho các kết quả về phép cộng, phép nhân, và chỉ thị SBN được trình bày trong các chương tiếp theo [1].

Chương 3

PHƯƠNG PHÁP ĐỀ XUẤT

3.1 Kỹ thuật Template Matching

Ý tưởng cốt lõi của phương pháp đề xuất là phân rã thuật toán thành các “mẫu” cục bộ, mỗi mẫu thao tác trên một khối bit nhỏ. Thay vì yêu cầu mạng nơ-ron học toàn bộ hàm phức tạp, ta chuyển bài toán thành việc nhận dạng và áp dụng các quy tắc cục bộ đơn giản.

3.1.1 Nguyên lý phân rã thuật toán

Xét một thuật toán hoạt động trên chuỗi bit $\mathbf{x} = (x_1, x_2, \dots, x_l)$ với l bit. Thuật toán được phân rã thành n khối, mỗi khối i chịu trách nhiệm tính toán một phần đầu ra dựa trên một tập con các bit đầu vào. Mỗi khối được đặc trưng bởi một hàm cục bộ f_i thao tác trên các bit trong phạm vi của nó.

Hàm khớp mẫu cục bộ: Mỗi hàm cục bộ $f_i : \{-1, +1\}^{b_i} \rightarrow \{-1, +1\}^{o_i}$ nhận đầu vào là b_i bit và trả về o_i bit đầu ra. Hàm này được định nghĩa thông qua một tập các mẫu (templates):

$$f_i(\mathbf{x}_{[S_i]}) = \bigvee_{j \in T_i} \mathbf{1}[\mathbf{x}_{[S_i]} = \mathbf{t}_j] \cdot \mathbf{o}_j \quad (3.1)$$

trong đó:

- S_i là tập chỉ số các bit đầu vào của khối i
- $\mathbf{x}_{[S_i]}$ là vector con của $\mathbf{x}$ tại các vị trí trong S_i
- T_i là tập các mẫu thuộc khối i
- $\mathbf{t}_j$ là vector mẫu đầu vào thứ j
- $\mathbf{o}_j$ là vector đầu ra tương ứng với mẫu j
- $\mathbf{1}[\cdot]$ là hàm chỉ thị

Hàm tổng hợp toàn cục: Đầu ra của thuật toán được tổng hợp từ các hàm cục bộ thông qua phép tuyển bitwise (bitwise disjunction):

$$f(\hat{\mathbf{x}}) = \bigvee_{i=1}^n f_i(\hat{\mathbf{x}}_{[S_i]}) \quad (3.2)$$

Phép tuyển $\vee$ hoạt động như phép “OR” trên các bit: nếu bất kỳ hàm cục bộ nào trả về bit 1 tại một vị trí, bit đầu ra tại vị trí đó sẽ là 1.

3.1.2 Cơ chế thực thi lặp

Nhiều thuật toán yêu cầu thực thi qua nhiều bước lặp, trong đó trạng thái của bước tiếp theo phụ thuộc vào kết quả của bước hiện tại. Phương pháp template matching hỗ trợ điều này thông qua cơ chế thực thi lặp:

$$\hat{\mathbf{x}}_{t+1} = f(\hat{\mathbf{x}}_t) \quad (3.3)$$

trong đó $\hat{\mathbf{x}}_t$ là trạng thái tại bước thứ t . Quá trình lặp tiếp tục cho đến khi đạt được trạng thái kết thúc hoặc sau một số bước cố định được xác định bởi thuật toán.

Điểm quan trọng là sau mỗi bước lặp, đầu ra được làm tròn về các giá trị rời rạc $\{-1, +1\}$ trước khi đưa vào bước tiếp theo. Điều này ngăn chặn tích lũy sai số và đảm bảo tính chính xác tuyệt đối.

3.2 Cấu trúc dữ liệu và Trục giao hóa

Để huấn luyện mạng nơ-ron học được các template, cần có chiến lược mã hóa phù hợp biến đổi các quy tắc thuật toán thành dữ liệu huấn luyện.

3.2.1 Mô hình “Bag of Rules”

Chiến lược mã hóa được sử dụng là gom tất cả các quy tắc cục bộ của thuật toán vào một không gian vector chung, gọi là mô hình “Bag of Rules”. Mỗi quy tắc cục bộ được biểu diễn như một cặp (đầu vào, đầu ra) trong không gian này.

Cụ thể, với thuật toán có n khối và tổng cộng M quy tắc cục bộ, tập huấn luyện có dạng:

$$\mathcal{D} = \{(\mathbf{q}_j, y_j)\}_{j=1}^M \quad (3.4)$$

trong đó $\mathbf{q}_j$ là vector mã hóa quy tắc thứ j và y_j là nhãn đầu ra tương ứng.

3.2.2 Mã hóa Block-Partitioned

Mỗi mẫu được mã hóa thành vector $\mathbf{q}_{ij}$ theo cấu trúc block-partitioned. Vector này có dạng:

$$\mathbf{q}_{ij} = (\underbrace{0, \dots, 0}_{\text{block 1}}, \dots, \underbrace{\mathbf{t}_{ij}}_{\text{block } i}, \dots, \underbrace{0, \dots, 0}_{\text{block } n}) \quad (3.5)$$

Ý nghĩa của cấu trúc này:

- Vector được chia thành n phân đoạn, mỗi phân đoạn tương ứng với một khối của thuật toán
- Chỉ phân đoạn thứ i chứa thông tin mẫu $\mathbf{t}_{ij}$, các phân đoạn khác đều bằng 0
- Điều này đảm bảo các mẫu thuộc các khối khác nhau có tích vô hướng bằng 0 với nhau

3.2.3 Trục giao hóa và Ma trận NTK

Tầm quan trọng của cấu trúc block-partitioned nằm ở việc nó tạo ra tính trục giao trong dữ liệu huấn luyện. Khi các mẫu thuộc các khối khác nhau trục giao với nhau, ma trận NTK có cấu trúc đặc biệt.

Gọi Θ là ma trận NTK trên tập huấn luyện. Với dữ liệu được mã hóa block-partitioned, ma trận này tiệm cận dạng:

$$\Theta \approx \alpha I_M + \beta \mathbf{1}\mathbf{1}^\top + (\text{nhiều bậc thấp}) \quad (3.6)$$

trong đó:

- α là hệ số đường chéo chính
- I_M là ma trận đơn vị kích thước $M \times M$
- $\beta \mathbf{1}\mathbf{1}^\top$ là ma trận hạng 1
- $\mathbf{1}$ là vector toàn 1

Cấu trúc “scaled identity + rank-1 noise” này cho phép phân tích chính xác hành vi của bộ dự đoán NTK và là nền tảng cho các định lý về khả năng học chính xác.

3.3 Định lý về bộ dự báo NTK

Phần này trình bày kết quả lý thuyết then chốt về hành vi của bộ dự đoán NTK khi áp dụng cho dữ liệu template matching.

3.3.1 Định lý 5.1: Bảo toàn thông tin dấu

Định lý 5.1 (Hành vi bộ dự báo NTK): Cho mạng nơ-ron hai lớp trong chế độ NTK ở giới hạn chiều rộng vô hạn được huấn luyện trên tập dữ liệu template $\mathcal{D}$. Gọi $\mu(\hat{\mathbf{x}})$ là kỳ vọng của phân phối giới hạn của đầu ra mạng tại điểm kiểm thử $\hat{\mathbf{x}}$. Khi đó, với mỗi bit đầu ra thứ i :

$$\text{sign}(\mu(\hat{\mathbf{x}})_i) = y_i^* \quad (3.7)$$

trong đó $y_i^* \in \{-1, +1\}$ là giá trị bit đầu ra đúng theo thuật toán.

3.3.2 Ý nghĩa của Định lý

Định lý 5.1 khẳng định rằng bộ dự đoán NTK bảo toàn thông tin về dấu của bit đầu ra:

- Nếu bit đầu ra đúng là $+1$, thì $\mu(\hat{\mathbf{x}})_i > 0$
- Nếu bit đầu ra đúng là -1 , thì $\mu(\hat{\mathbf{x}})_i < 0$

Điều này có ý nghĩa thực tiễn quan trọng: mặc dù đầu ra thô của mạng là một giá trị thực (có thể không chính xác bằng ± 1), việc áp dụng hàm làm tròn sẽ khôi phục giá trị chính xác:

$$y_{\text{output}} = \text{round}(\mu(\hat{\mathbf{x}})_i) = \text{sign}(\mu(\hat{\mathbf{x}})_i) = y_i^* \quad (3.8)$$

Kết quả này cho phép mạng nơ-ron đạt được độ chính xác tuyệt đối thay vì chỉ xấp xỉ, giải quyết thách thức cốt lõi về việc học các hàm rời rạc.

3.4 Kiểm soát tương quan không mong muốn

Định lý 5.1 không tự động đúng cho mọi thuật toán. Để đảm bảo tính đúng đắn, cần kiểm soát các “tương quan không mong muốn” trong dữ liệu huấn luyện.

3.4.1 Assumption 5.1: Điều kiện về tương quan

Phân tích đầu ra của bộ dự đoán NTK cho thấy:

$$\mu(\hat{\mathbf{x}})_i = w^1 \cdot y_i^* + w^0 \cdot \sum_{j \in U_i} y_j \quad (3.9)$$

trong đó:

- $w^1 > 0$ là *trọng số tín hiệu*: Đóng góp từ template khớp chính xác
- w^0 là *trọng số nhiễu*: Đóng góp từ các template có tương quan khác không
- U_i là tập các chỉ số của các template không khớp nhưng có tích vô hướng khác không với đầu vào
- y_j là nhãn của template thứ j trong tập nhiễu

Assumption 5.1: Gọi $C_i = |U_i|$ là số lượng tương quan không mong muốn tại bit thứ i . Điều kiện để đầu ra có dấu đúng là:

$$C_i < -\frac{w^1}{w^0} \quad (3.10)$$

Lưu ý rằng $w^0 < 0$ trong các trường hợp được xét, do đó $-w^1/w^0 > 0$.

3.4.2 Phân tích trường hợp xấu nhất

Trong trường hợp xấu nhất, tất cả các tương quan không mong muốn đều “đồng pha” (có cùng dấu với nhiễu), tổng nhiễu có thể đạt:

$$\left| \sum_{j \in U_i} y_j \right| \leq C_i \quad (3.11)$$

Để đảm bảo thành phần tín hiệu lấn át nhiễu:

$$|w^1 \cdot y_i^*| > |w^0 \cdot C_i| \Rightarrow |w^1| > C_i \cdot |w^0| \Rightarrow C_i < \frac{|w^1|}{|w^0|} \quad (3.12)$$

3.4.3 Kết quả cho các thuật toán cụ thể

Các tác giả đã chứng minh rằng các thuật toán được xét đều thỏa mãn Assumption 5.1:

- **Phép hoán vị:** $C_i = 0$ (không có tương quan không mong muốn)

- **Phép cộng:** $C_i \leq 1$ (tối đa 1 tương quan không mong muốn)
- **Phép nhân:** $C_i = O(\log l)$ với l là số bit
- **SBN:** $C_i = O(\log l)$

Với thiết kế template phù hợp, tỷ lệ $|w^1|/|w^0|$ tăng nhanh hơn C_i khi kích thước bài toán tăng, đảm bảo điều kiện được thỏa mãn.

3.5 Triển khai thuật toán cụ thể

Phần này trình bày cách áp dụng khung template matching cho từng thuật toán cụ thể.

3.5.1 Phép hoán vị

Phép hoán vị $\pi : [l] \rightarrow [l]$ ánh xạ vị trí bit i sang vị trí mới $\pi(i)$. Đây là thuật toán đơn giản nhất để minh họa phương pháp.

Mã hóa template: Mỗi bit được xử lý độc lập. Với bit thứ i , có 2 template:

- Template 1: Đầu vào $x_i = +1 \Rightarrow$ Đầu ra tại vị trí $\pi(i)$ là $+1$
- Template 2: Đầu vào $x_i = -1 \Rightarrow$ Đầu ra tại vị trí $\pi(i)$ là -1

Số lượng template: $2l$ template tổng cộng.

Tương quan không mong muốn: Vì mỗi template chỉ phụ thuộc vào một bit đầu vào duy nhất và các bit được mã hóa trong các block riêng biệt, không có tương quan không mong muốn ($C_i = 0$).

3.5.2 Phép cộng

Phép cộng hai số nhị phân l -bit được triển khai thông qua mô phỏng bộ cộng ripple-carry. Thuật toán gồm 2 giai đoạn:

Giai đoạn 1 - Cộng bitwise: Tính tổng từng bit và bit nhớ cục bộ:

$$s_i = a_i \oplus b_i \oplus c_{i-1} \quad (3.13)$$

$$c_i = \text{MAJ}(a_i, b_i, c_{i-1}) \quad (3.14)$$

trong đó $\oplus$ là phép XOR, MAJ là hàm majority, và c_{i-1} là bit nhớ từ vị trí trước.

Giai đoạn 2 - Lan truyền bit nhớ: Bit nhớ được lan truyền từ phải sang trái. Quá trình này yêu cầu $O(\log l)$ bước lặp do kỹ thuật parallel prefix.

Template cho mỗi vị trí: Mỗi vị trí bit i có $2^3 = 8$ template (tương ứng với 8 tổ hợp của a_i, b_i, c_{i-1}).

Tương quan không mong muốn: Tối đa $C_i = 1$, xảy ra khi hai template có cùng giá trị (a_i, b_i) nhưng khác c_{i-1} .

3.5.3 Phép nhân

Phép nhân hai số nhị phân l -bit sử dụng thuật toán shift-and-add cổ điển. Thuật toán lặp qua từng bit của số nhân (multiplier) và thực hiện:

1. Kiểm tra bit thấp nhất (LSB) của multiplier
2. Nếu $\text{LSB} = 1$: Cộng multiplicand vào kết quả tích lũy
3. Dịch trái multiplicand và dịch phải multiplier

Số bước lặp: l bước cho l -bit multiplication.

Mã hóa template: Mỗi bước lặp được mã hóa thành các template riêng, bao gồm:

- Template kiểm tra LSB
- Template cộng có điều kiện
- Template dịch bit

Tương quan không mong muốn: $C_i = O(\log l)$, phát sinh từ sự chồng chéo giữa các bước lặp.

3.5.4 Chỉ thị SBN

SBN là một chỉ thị máy tính đơn giản nhưng đủ mạnh để đạt tính đầy đủ Turing. Chỉ thị $\text{SBN}(A, B, C)$ thực hiện:

1. Tính $\text{Mem}[A] \leftarrow \text{Mem}[A] - \text{Mem}[B]$
2. Nếu kết quả âm, nhảy đến địa chỉ C ; ngược lại, tiếp tục tuần tự

extbfTính đầy đủ Turing: SBN được chứng minh là đầy đủ Turing, nghĩa là bất kỳ hàm tính toán được nào cũng có thể được biểu diễn bằng một chuỗi hữu hạn các chỉ thị SBN.

Mã hóa template cho SBN: Bao gồm các thành phần:

- Template cho phép trừ: Tương tự phép cộng với số bù 2
- Template kiểm tra dấu kết quả
- Template cập nhật program counter

Ý nghĩa: Việc chứng minh mạng nơ-ron có thể học chính xác SBN đồng nghĩa với việc mạng nơ-ron có khả năng, về nguyên tắc, học thực thi bất kỳ thuật toán nào có thể được mô tả bằng chương trình máy tính.

3.6 Độ phức tạp tập hợp

Mặc dù lý thuyết NTK đảm bảo tính đúng đắn về kỳ vọng, phương sai của từng mô hình riêng lẻ có thể ảnh hưởng đến kết quả thực tế. Để giảm thiểu rủi ro này, phương pháp sử dụng kỹ thuật ensemble.

3.6.1 Tại sao cần Ensemble?

Trong thực tế, với mạng có chiều rộng hữu hạn m , đầu ra có phân phối:

$$F(\hat{\mathbf{x}}) \sim \mathcal{N}(\mu(\hat{\mathbf{x}}), \sigma^2(\hat{\mathbf{x}})) \quad (3.15)$$

trong đó σ^2 là phương sai. Ngay cả khi μ có dấu đúng, phương sai lớn có thể khiến một số lần thực thi cho kết quả sai.

Kỹ thuật ensemble giải quyết vấn đề này bằng cách huấn luyện N mô hình độc lập và lấy trung bình:

$$\bar{F}(\hat{\mathbf{x}}) = \frac{1}{N} \sum_{k=1}^N F_k(\hat{\mathbf{x}}) \quad (3.16)$$

Phương sai của đầu ra trung bình giảm theo $1/N$:

$$\text{Var}[\bar{F}] = \frac{\sigma^2}{N} \quad (3.17)$$

3.6.2 Công thức tính số lượng mô hình tối thiểu

Để đảm bảo xác suất lỗi không vượt quá δ , số lượng mô hình N cần thỏa mãn (Công thức (8) trong bài báo gốc):

$$N \geq \frac{2\sigma^2}{\mu^2} \ln \left(\frac{2}{\delta} \right) \quad (3.18)$$

trong đó:

- μ là kỳ vọng của đầu ra (đã được chứng minh có dấu đúng)
- σ^2 là phương sai của từng mô hình
- δ là xác suất lỗi cho phép

Công thức này dựa trên bất đẳng thức Hoeffding cho tổng các biến ngẫu nhiên độc lập.

3.6.3 Độ phức tạp theo kích thước bài toán

Phân tích cho thấy tỷ lệ σ^2/μ^2 tăng theo $O(l^2)$ với l là số bit của bài toán. Do đó, độ phức tạp ensemble là:

$$N = O(l^2 \log(1/\delta)) \quad (3.19)$$

Kết quả này có ý nghĩa thực tiễn quan trọng:

- Độ phức tạp chỉ tăng đa thức theo kích thước bài toán
- Với $l = 64$ bit (số nguyên 64-bit), cần khoảng $N = O(4096)$ mô hình
- Các mô hình có thể được huấn luyện song song, phù hợp với phần cứng GPU hiện đại

Kết luận, phương pháp đề xuất có khả năng mở rộng tốt cho các bài toán thực tế, không bị giới hạn bởi sự tăng trưởng theo hàm mũ thường gặp trong các phương pháp học máy truyền thống cho bài toán tổ hợp.

Chương 4

THỰC NGHIỆM VÀ PHÂN TÍCH

Chương này trình bày các thực nghiệm được thực hiện để xác minh tính đúng đắn của khung lý thuyết đã đề xuất. Các kết quả thực nghiệm được đối chiếu với các dự đoán lý thuyết, cho thấy sự phù hợp cao giữa lý thuyết và thực tiễn.

4.1 Thiết lập thực nghiệm

4.1.1 Kiến trúc mạng

Kiến trúc mạng được sử dụng trong thực nghiệm là mạng nơ-ron hai lớp kết nối đầy đủ với các đặc điểm sau:

- **Hàm kích hoạt:** ReLU được sử dụng cho lớp ẩn
- **Bias:** Mạng không sử dụng bias trong các lớp
- **Chiều rộng lớp ẩn:** $n_h = 50,000$ nơ-ron ẩn, đủ lớn để tiệm cận chế độ NTK ở giới hạn chiều rộng vô hạn

Cấu trúc này được chọn để phù hợp với các giả định trong phân tích lý thuyết về NTK, đồng thời vẫn khả thi về mặt tính toán.

4.1.2 Tham số khởi tạo

Các trọng số được khởi tạo theo *NTK parametrization* với các đặc điểm:

- Trọng số lớp ẩn: $W^{(1)} \sim \mathcal{N}(0, 1)$ (phân phối Gaussian chuẩn)
- Trọng số lớp đầu ra: $W^{(2)} \sim \mathcal{N}(0, 1/n_h)$ (chia tỷ lệ theo chiều rộng)

Việc sử dụng NTK parametrization đảm bảo rằng khi $n_h \rightarrow \infty$, động lực học huấn luyện được mô tả chính xác bởi kernel NTK cố định.

4.1.3 Cấu hình huấn luyện

- **Thuật toán tối ưu:** Hạ gradient toàn bộ dữ liệu được sử dụng thay vì huấn luyện theo lô nhỏ, đảm bảo gradient được tính trên toàn bộ tập huấn luyện

- **Tốc độ học:** Được chọn đủ nhỏ để đảm bảo hội tụ ổn định trong chế độ huấn luyện lười
- **Tiêu chí dừng:** Huấn luyện cho đến khi loss giảm đến ngưỡng đủ nhỏ hoặc đạt số epoch tối đa

4.1.4 Đối tượng kiểm thử

Các thực nghiệm được thực hiện trên các chuỗi bit có độ dài l thay đổi:

- Phạm vi độ dài bit: $l \in \{2, 3, 4, \dots, 30\}$
- Với mỗi giá trị l , tập huấn luyện bao gồm tất cả các template cần thiết cho thuật toán tương ứng
- Tập kiểm tra gồm các đầu vào ngẫu nhiên để đánh giá khả năng tổng quát hóa

4.2 Xác minh lý thuyết

Để xác minh tính đúng đắn của Định lý 5.1 một cách trực tiếp mà không bị ảnh hưởng bởi các yếu tố nhiễu trong huấn luyện thực tế, nghiên cứu sử dụng phương pháp tính toán NTK predictor.

4.2.1 Phương pháp xác minh

Thay vì huấn luyện mạng nơ-ron thực tế, nghiên cứu sử dụng thư viện *Neural Tangents* để tính toán trực tiếp bộ dự báo NTK. Phương pháp này có các ưu điểm:

- **Tính chính xác:** Loại bỏ các yếu tố nhiễu từ quá trình huấn luyện (tốc độ học, số epoch, v.v.)
- **Tính hiệu quả:** Tính toán dạng tường minh thay vì tối ưu hóa lặp
- **Xác minh lý thuyết:** Kiểm tra trực tiếp các dự đoán của Định lý 5.1 trong chế độ NTK lý tưởng

Cụ thể, với tập huấn luyện $\{(\mathbf{x}_i, y_i)\}_{i=1}^M$ và điểm test $\hat{\mathbf{x}}$, bộ dự báo NTK được tính:

$$\mu(\hat{\mathbf{x}}) = \Theta(\hat{\mathbf{x}}, X) \cdot \Theta(X, X)^{-1} \cdot Y \quad (4.1)$$

trong đó Θ là ma trận kernel NTK.

4.2.2 Kết quả trên phép cộng và nhân

Các thực nghiệm xác minh được thực hiện trên hai thuật toán số học cơ bản: phép cộng và phép nhân nhị phân.

Kết quả chính:

- Với cả phép cộng và phép nhân nhị phân, bộ dự báo NTK cho kết quả **chính xác tuyệt đối** sau khi làm tròn

- Độ chính xác 100% được đạt được cho các chuỗi bit có độ dài lên đến $l = 10$
- Mọi bit đầu ra đều có dấu đúng, xác nhận Định lý 5.1: $\text{sign}(\mu(\hat{\mathbf{x}})_i) = y_i^*$

Ý nghĩa: Kết quả này xác nhận rằng khung lý thuyết template matching kết hợp với phân tích NTK có thể dự đoán chính xác hành vi của mạng nơ-ron trên các bài toán số học nhị phân.

4.2.3 So sánh phương sai và kỳ vọng

Một phần quan trọng của phân tích lý thuyết là mối quan hệ giữa phương sai $\sigma^2(\hat{\mathbf{x}})$ và kỳ vọng $\mu(\hat{\mathbf{x}})$ của đầu ra mạng. Nghiên cứu phân tích tỷ lệ:

$$R_i = \frac{\sigma^2(\hat{\mathbf{x}})_i}{\mu^2(\hat{\mathbf{x}})_i} \quad (4.2)$$

cho mỗi bit đầu ra thứ i .

Dự đoán lý thuyết: Theo phân tích trong Chương 3, tỷ lệ σ^2/μ^2 được dự đoán tăng tuyến tính theo kích thước đầu vào k' (số lượng thành phần khác không trong vector đầu vào sparse):

$$\frac{\sigma^2}{\mu^2} = O(k') \quad (4.3)$$

Kết quả thực nghiệm:

- Đồ thị tỷ lệ σ^2/μ^2 theo k' cho thấy xu hướng tăng tuyến tính rõ ràng
- Độ dốc của đường xu hướng phù hợp với hệ số được dự đoán từ lý thuyết
- Sự phù hợp này xác nhận rằng công thức tính ensemble complexity trong Công thức (8) là chính xác

Nhận xét: Sự phù hợp giữa dự đoán lý thuyết và kết quả thực nghiệm xác nhận rằng ensemble complexity $N = O(l^2 \log(1/\delta))$ là ước lượng đáng tin cậy cho số lượng mô hình cần thiết.

4.3 Thực nghiệm huấn luyện

Phần này trình bày các thực nghiệm huấn luyện thực tế (training experiments) để đánh giá khả năng học của mạng nơ-ron trong điều kiện thực tiễn.

4.3.1 Lựa chọn bài toán

Nghiên cứu chọn bài toán **Hoán vị** làm đối tượng thực nghiệm chính với các lý do:

- **Ensemble complexity thấp hơn:** So với phép cộng và nhân, phép hoán vị có $C_i = 0$ (không có tương quan không mong muốn), dẫn đến tỷ lệ σ^2/μ^2 nhỏ hơn
- **Tài nguyên tính toán:** Số lượng mô hình ensemble cần thiết nằm trong khả năng của phần cứng hiện có
- **Tính đại diện:** Mặc dù đơn giản hơn, hoán vị vẫn đủ để chứng minh tính đúng đắn của lý thuyết về ensemble và khả năng đạt độ chính xác tuyệt đối

4.3.2 Bài toán Hoán vị

Định nghĩa bài toán: Cho hoán vị $\pi : [n] \rightarrow [n]$ ngẫu nhiên, mạng nơ-ron được huấn luyện để học ánh xạ này trên các vector cơ sở chuẩn.

Thiết lập thực nghiệm:

- **Tập huấn luyện:** Bao gồm $2n$ template tương ứng với tất cả các ánh xạ vị trí của hoán vị
- **Tập kiểm tra:** 1000 vector kiểm thử được sinh ngẫu nhiên, mỗi vector có $n_{\hat{x}} = 2$ phần tử khác không
- **Độ chính xác:** Được tính là tỷ lệ các vector kiểm tra mà tất cả các bit đầu ra đều đúng sau khi làm tròn

Điều kiện thành công: Vector kiểm tra được coi là đúng nếu và chỉ nếu:

$$\forall i : \text{sign}(\bar{F}(\hat{\mathbf{x}})_i) = y_i^* \quad (4.4)$$

trong đó $\bar{F}$ là đầu ra trung bình của ensemble.

4.3.3 Độ chính xác theo kích thước Ensemble

Mục tiêu thực nghiệm: So sánh số lượng mô hình thực tế cần thiết để đạt độ chính xác mục tiêu (90%, 95%, 99%) với ngưỡng lý thuyết từ Công thức (8):

$$N_{\text{theory}} \geq \frac{2\sigma^2}{\mu^2} \ln \left(\frac{2}{\delta} \right) \quad (4.5)$$

Kết quả Thực nghiệm 1:

- Với các giá trị k (kích thước đầu vào sparse) khác nhau, đường độ chính xác thực nghiệm được so sánh với ngưỡng lý thuyết
- **Quan sát chính:** Số lượng mô hình thực tế cần thiết **luôn nhỏ hơn** ngưỡng chặn lý thuyết
- Điều này xác nhận rằng ngưỡng lý thuyết có tính chất bảo thủ, đảm bảo an toàn

Kết quả Thực nghiệm 2:

- Khi kích thước ensemble tăng lên, độ chính xác tiến dần đến 100%
- Với k lớn hơn, cần nhiều mô hình hơn để đạt cùng mức độ chính xác
- Xu hướng này phù hợp với dự đoán lý thuyết: ensemble complexity tăng theo $O(k^2)$

4.3.4 Phân tích kết quả

Tính bảo thủ của ngưỡng lý thuyết:

Ngưỡng chặn lý thuyết được xây dựng dựa trên:

- Bất đẳng thức Hoeffding cho tổng các biến ngẫu nhiên con-Gaussian
- Kỹ thuật *union bound* để đảm bảo tất cả các bit đầu ra đều đúng đồng thời

Do sử dụng các bất đẳng thức theo trường hợp xấu nhất, ngưỡng này có tính chất **bảo thủ cao**. Trong thực tế:

- Các phân phối thực tế thường “tốt hơn” giả định theo trường hợp xấu nhất
- Tương quan giữa các sai số ở các bit khác nhau có thể có lợi
- Kết quả là số mô hình thực tế cần thiết thường nhỏ hơn đáng kể so với ngưỡng lý thuyết

Ý nghĩa thực tiễn:

Kết quả thực nghiệm xác nhận hai điều quan trọng:

1. **Tính đúng đắn của lý thuyết:** Ngưỡng lý thuyết luôn là chặn trên hợp lệ cho số mô hình cần thiết
2. **Khả năng thực tiễn:** Trong ứng dụng thực tế, có thể sử dụng ít mô hình hơn so với ngưỡng lý thuyết mà vẫn đạt độ chính xác cao

4.4 Tổng hợp và Đánh giá

Kết nối giữa lý thuyết và thực nghiệm:

Các thực nghiệm trong chương này đã xác nhận ba dự đoán lý thuyết chính:

1. **Định lý 5.1:** Bộ dự báo NTK bảo toàn thông tin dấu, cho phép đạt độ chính xác tuyệt đối sau làm tròn
2. **Mối quan hệ σ^2/μ^2 :** Tỷ lệ này tăng tuyến tính theo kích thước đầu vào, phù hợp với phân tích lý thuyết
3. **Ensemble complexity:** Công thức (8) cung cấp ngưỡng chặn trên hợp lệ và có tính bảo thủ

Kết luận về khả năng học chính xác:

Kết quả thực nghiệm cung cấp bằng chứng mạnh mẽ rằng:

- Mạng nơ-ron **có khả năng** học thực thi chính xác các thuật toán số học và logic
- Phương pháp template matching với kỹ thuật ensemble là **khả thi về mặt tính toán**
- Lý thuyết NTK cung cấp **khung phân tích đáng tin cậy** cho việc thiết kế và đánh giá các hệ thống như vậy

Những kết quả này mở ra hướng nghiên cứu về việc xây dựng các mạng nơ-ron có khả năng thực thi chính xác, không chỉ xấp xỉ, các hàm tính toán phức tạp.

Chương 5

KẾT LUẬN

5.1 Tổng kết nghiên cứu

Nghiên cứu này đã giải quyết thành công câu hỏi cốt lõi về khả năng học và thực thi chính xác tuyệt đối các thuật toán nhị phân của mạng nơ-ron hai lớp trong giới hạn chiều rộng vô hạn ($n_h \rightarrow \infty$). Thông qua việc kết hợp kỹ thuật template matching với phân tích Neural Tangent Kernel (NTK), nghiên cứu đã đưa ra các chứng minh lý thuyết vững chắc được xác nhận bởi thực nghiệm.

Các kết quả chính đạt được:

1. **Chứng minh *exact learnability*:** Nghiên cứu đã chứng minh tính học được chính xác cho bốn nhóm tác vụ cơ bản:
 - Phép hoán vị
 - Phép cộng nhị phân
 - Phép nhân nhị phân
 - Chỉ thị *SBN*
2. **Hiệu quả dữ liệu vượt trội:** Phương pháp chỉ yêu cầu số lượng mẫu huấn luyện theo bậc logarit so với kích thước không gian đầu vào ($O(\log 2^l) = O(l)$ thay vì $O(2^l)$), giúp việc huấn luyện khả thi cho các bài toán với số bit lớn.
3. **Vai trò của lý thuyết NTK:** Khung toán học NTK cho phép dự báo chính xác hành vi của mạng mà không cần huấn luyện thực tế, cung cấp công cụ phân tích mạnh mẽ cho việc thiết kế và đánh giá hệ thống.

5.2 Ý nghĩa của nghiên cứu

Tính phổ quát của kết quả:

Việc thực thi thành công chỉ thị *SBN*—vốn được chứng minh là đầy đủ Turing—có ý nghĩa sâu sắc: về nguyên tắc, mạng nơ-ron có khả năng thực thi mọi hàm số tính toán được. Điều này thiết lập cầu nối lý thuyết giữa khả năng biểu diễn của mạng nơ-ron và lý thuyết tính toán cổ điển.

Đóng góp lý thuyết:

Đây là chứng minh dựa trên NTK đầu tiên cho việc thực thi thuật toán chính xác. Khác với các nghiên cứu trước đó chỉ tập trung vào xấp xỉ các hàm trơn, nghiên cứu này chứng minh rằng mạng nơ-ron có thể học các hàm rời rạc với độ chính xác tuyệt đối thông qua cơ chế làm tròn.

5.3 Hạn chế của phương pháp

Mặc dù đạt được các kết quả lý thuyết quan trọng, phương pháp vẫn tồn tại một số hạn chế cần được nhận thức rõ:

Tính trực giao của dữ liệu:

Phương pháp phụ thuộc mạnh vào giả định rằng dữ liệu huấn luyện được trực giao hóa và cấu trúc hóa theo dạng block-partitioned. Trong các tập dữ liệu tự nhiên, tính chất trực giao này khó đạt được một cách tự động.

Bộ nhớ hữu hạn:

Mô hình bị giới hạn bởi kích thước đầu vào cố định. Khi thay đổi kích thước bit—ví dụ từ 10 bit lên 20 bit—mạng cần được huấn luyện lại hoàn toàn do kiến trúc không tự động mở rộng. Điều này hạn chế tính linh hoạt trong ứng dụng thực tế.

Khả năng tổng quát hóa độ dài:

Phương pháp chưa đạt được tổng quát hóa theo độ dài—khả năng tổng quát hóa cho các chuỗi đầu vào dài hơn so với dữ liệu huấn luyện. Đây là một hạn chế so với các kiến trúc RNN hay Transformer hiện đại, vốn có khả năng xử lý trường hợp ngoài phân phối về độ dài một cách tự nhiên hơn.

5.4 Hướng phát triển tương lai

Dựa trên các kết quả và hạn chế đã phân tích, một số hướng nghiên cứu tiềm năng được đề xuất:

1. **Dữ liệu có tương quan:** Nghiên cứu khả năng *exact learnability* khi dữ liệu huấn luyện có tương quan thay vì tuân theo giả định trực giao nghiêm ngặt.
2. **Kiến trúc có độ dài thay đổi:** Mở rộng khung lý thuyết NTK sang các kiến trúc có khả năng xử lý đầu vào với độ dài thay đổi như Transformer, RNN, hoặc Graph Neural Networks (GNN) trên đồ thị có bậc bị chặn.
3. **Học chỉ thị từ dữ liệu:** Khám phá khả năng mạng tự suy diễn ra các chỉ thị nguyên thủy từ các cặp đầu vào-đầu ra thực tế, thay vì được cung cấp sẵn các template như trong phương pháp hiện tại.

Kết luận: Nghiên cứu này đã chứng minh rằng ranh giới giữa tính toán logic-thuật toán và học máy không phải là không thể vượt qua. Việc mạng nơ-ron có thể học thực thi chính xác các phép toán cơ bản—và thậm chí là các chỉ thị đầy đủ Turing—mở ra triển vọng về một thể hệ hệ thống AI mới: kết hợp khả năng học từ dữ liệu của mạng nơ-ron với độ tin cậy và tính chính xác của tính toán thuật toán truyền thống.

Tài liệu tham khảo

Tài liệu tham khảo

- [1] Artur Back de Luca, George Giapitzakis, and Kimon Fountoulakis, “Learning to Add, Multiply, and Execute Algorithmic Instructions Exactly with Neural Networks”, arXiv:2502.16763, 2025.
- [2] Arthur Jacot, Franck Gabriel, and Clément Hongler, “Neural Tangent Kernel: Convergence and Generalization in Neural Networks”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [3] Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel S. Schoenholz, Jeffrey Pennington, and Jascha Sohl-Dickstein, “Deep Neural Networks as Gaussian Processes”, *International Conference on Learning Representations (ICLR)*, 2018.
- [4] Greg Yang, “Tensor Programs I: Wide Feedforward or Recurrent Neural Networks of Any Architecture are Gaussian Processes”, arXiv:1910.12478, 2019.
- [5] Lénaïc Chizat and Francis Bach, “On the Global Convergence of Gradient Descent for Over-Parameterized Models using Optimal Transport”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. (Bản mở rộng: arXiv:1805.09545, 2019.)
- [6] Ali Rahimi and Benjamin Recht, “Random Features for Large-Scale Kernel Machines”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2008.
- [7] Christopher K. I. Williams, “Computing with Infinite Networks”, *Advances in Neural Information Processing Systems (NeurIPS)*, 1997.
- [8] Youngmin Cho and Lawrence K. Saul, “Kernel Methods for Deep Learning”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2009.
- [9] Bernhard Schölkopf and Alexander J. Smola, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*, MIT Press, 2002.
- [10] Sanjeev Arora, Simon S. Du, Wei Hu, Zhiyuan Li, Ruslan Salakhutdinov, and Ruosong Wang, “On Exact Computation with an Infinitely Wide Neural Net”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [11] Simon S. Du, Jason D. Lee, Haochuan Li, Liwei Wang, and Igor S. Dhillon, “Gradient Descent Finds Global Minima of Deep Neural Networks”, *International Conference on Machine Learning (ICML)*, 2019.
- [12] Song Mei, Andrea Montanari, and Phan-Minh Nguyen, “A Mean Field View of the Landscape of Two-Layer Neural Networks”, *Proceedings of the National Academy of Sciences (PNAS)*, 115(33):E7665–E7671, 2018.

- [13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin, “Attention Is All You Need”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [14] Wojciech Zaremba and Ilya Sutskever, “Learning to Execute”, arXiv:1410.4615, 2014.
- [15] Alex Graves, Greg Wayne, and Ivo Danihelka, “Neural Turing Machines”, arXiv:1410.5401, 2014.
- [16] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis, “Hybrid Computing Using a Neural Network with Dynamic External Memory”, *Nature*, 538:471–476, 2016.
- [17] Łukasz Kaiser and Ilya Sutskever, “Neural GPUs Learn Algorithms”, *International Conference on Learning Representations (ICLR)*, 2016 (arXiv:1511.08228, 2015).
- [18] Scott Reed and Nando de Freitas, “Neural Programmer-Interpreters”, *International Conference on Learning Representations (ICLR)*, 2016.
- [19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, *NAACL-HLT*, 2019 (arXiv:1810.04805, 2018).
- [20] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al., “Language Models are Few-Shot Learners”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [21] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Łukasz Kaiser, Matthias Plappert, Fotios Petroni, and others, “Training Verifiers to Solve Math Word Problems”, arXiv:2110.14168, 2021. (Dataset GSM8K.)
- [22] Chulhee Yun, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank J. Reddi, and Sanjiv Kumar, “Are Transformers Universal Approximators of Sequence-to-Sequence Functions?”, *International Conference on Learning Representations (ICLR)*, 2020.
- [23] Jorge Pérez, Pablo Barceló, and Javier María Tur, “The Computational Power of Transformers”, *International Conference on Learning Representations (ICLR)*, 2021.
- [24] John E. Hopcroft and Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, Addison-Wesley, 1979.
- [25] Marvin Minsky, *Computation: Finite and Infinite Machines*, Prentice-Hall, 1967.
- [26] Hava T. Siegelmann and Eduardo D. Sontag, “On the Computational Power of Neural Nets”, *Journal of Computer and System Sciences*, 50(1):132–150, 1995.
- [27] Alan M. Turing, “On Computable Numbers, with an Application to the Entscheidungsproblem”, *Proceedings of the London Mathematical Society*, 42(2):230–265, 1936.

- [28] Stephen A. Cook, “The Complexity of Theorem-Proving Procedures”, *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC)*, 1971.

Phụ lục A

Chứng minh toán học bổ trợ

Phụ lục này trình bày các kết quả toán học nền tảng được sử dụng trong phân tích lý thuyết của nghiên cứu. Mỗi kết quả được giải thích về mặt vai trò và ý nghĩa đối với các chứng minh chính.

A.1 Các kết quả đại số tuyến tính

A.1.1 Định lý Sherman-Morrison

Định lý *Sherman-Morrison* (1950) là công cụ quan trọng để tính nghịch đảo của ma trận khi có sự thay đổi hạng 1 (rank-1 update). Đây là nền tảng cho việc phân tích ma trận NTK có cấu trúc “scaled identity + rank-1 noise”.

Định lý (Sherman-Morrison): Cho A là ma trận khả nghịch kích thước $n \times n$ và u, v là các vector cột kích thước n . Nếu $1 + v^\top A^{-1}u \neq 0$, thì:

$$(A + uv^\top)^{-1} = A^{-1} - \frac{A^{-1}uv^\top A^{-1}}{1 + v^\top A^{-1}u} \quad (\text{A.1})$$

Vai trò trong nghiên cứu:

Ma trận NTK trên tập huấn luyện có dạng:

$$\Theta = \alpha I_M + \beta \mathbf{1}\mathbf{1}^\top + (\text{nhiều bậc cao}) \quad (\text{A.2})$$

trong đó $\mathbf{1}\mathbf{1}^\top$ là ma trận rank-1. Áp dụng Sherman-Morrison với $A = \alpha I_M$, $u = v = \sqrt{\beta}\mathbf{1}$:

$$\Theta^{-1} \approx (\alpha I_M + \beta \mathbf{1}\mathbf{1}^\top)^{-1} \quad (\text{A.3})$$

$$= \frac{1}{\alpha} I_M - \frac{\frac{1}{\alpha^2} \beta \mathbf{1}\mathbf{1}^\top}{1 + \frac{\beta M}{\alpha}} \quad (\text{A.4})$$

$$= \frac{1}{\alpha} I_M - \frac{\beta}{\alpha(\alpha + \beta M)} \mathbf{1}\mathbf{1}^\top \quad (\text{A.5})$$

Công thức này cho phép tính closed-form của Θ^{-1} mà không cần thuật toán nghịch đảo ma trận $O(M^3)$, đồng thời làm rõ cấu trúc của bộ dự báo NTK.

A.2 Các kết quả xác suất

A.2.1 Bổ đề tập trung Gaussian

Bổ đề tập trung (concentration bound) cho phân phối Gaussian là cơ sở để xác định số lượng mô hình ensemble cần thiết.

Bổ đề A.1 (Gaussian Concentration): Cho $X_1, X_2, \dots, X_n$ là các biến ngẫu nhiên độc lập, cùng phân phối $\mathcal{N}(\mu, \sigma^2)$. Gọi $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$ là trung bình mẫu. Khi đó, với mọi $t > 0$:

$$\mathbb{P}\{|\bar{X}_n - \mu| \geq t\} \leq 2 \exp\left(-\frac{nt^2}{2\sigma^2}\right) \quad (\text{A.6})$$

Ứng dụng trong tính ensemble size:

Đầu ra của mỗi mô hình trong ensemble tuân theo phân phối Gaussian với kỳ vọng μ (có dấu đúng) và phương sai σ^2 . Để đảm bảo trung bình ensemble $\bar{F}$ có dấu đúng với xác suất ít nhất $1 - \delta$, ta cần:

$$\mathbb{P}\{|\bar{F} - \mu| \geq |\mu|\} \leq \delta \quad (\text{A.7})$$

Áp dụng Bổ đề A.1 với $t = |\mu|$:

$$2 \exp\left(-\frac{N\mu^2}{2\sigma^2}\right) \leq \delta \quad (\text{A.8})$$

$$N \geq \frac{2\sigma^2}{\mu^2} \ln\left(\frac{2}{\delta}\right) \quad (\text{A.9})$$

Đây chính là Công thức (8) trong main paper, cung cấp ngưỡng chặn trên cho số mô hình cần thiết.

A.3 Chi tiết tính toán bộ dự báo NTK

Phần này trình bày công thức tính các trọng số quyết định trong đầu ra của bộ dự báo NTK.

A.3.1 Phân rã đầu ra NTK

Đầu ra của bộ dự báo NTK tại điểm test $\hat{\mathbf{x}}$ có thể được phân rã:

$$\mu(\hat{\mathbf{x}})_i = \sum_{j=1}^M w_j(\hat{\mathbf{x}}) \cdot y_j \quad (\text{A.10})$$

trong đó $w_j(\hat{\mathbf{x}}) = [\Theta(\hat{\mathbf{x}}, X) \cdot \Theta(X, X)^{-1}]_j$ là trọng số đóng góp của mẫu huấn luyện thứ j .

A.3.2 Trọng số tín hiệu và nhiễu

Với cấu trúc block-partitioned của dữ liệu, các trọng số được phân loại thành hai nhóm:

Trọng số tín hiệu $w^1(\hat{\mathbf{x}})$: Áp dụng cho template khớp chính xác (tích vô hướng bằng 1):

$$w^1(\hat{\mathbf{x}}) = \frac{1}{\alpha} - \frac{\beta}{(\alpha + \beta M)\alpha} \cdot \sum_{j \in \text{match}} y_j \quad (\text{A.11})$$

Trọng số nhiễu $w^0(\hat{\mathbf{x}})$: Áp dụng cho các template có tương quan không hoàn toàn (tích vô hướng khác 0 và 1):

$$w^0(\hat{\mathbf{x}}) = \frac{\gamma}{\alpha} - \frac{\beta\gamma}{(\alpha + \beta M)\alpha} \quad (\text{A.12})$$

trong đó γ là giá trị tích vô hướng giữa $\hat{\mathbf{x}}$ và template tương ứng.

A.3.3 Điều kiện để dấu đúng

Đầu ra có dấu đúng khi thành phần tín hiệu lớn hơn thành phần nhiễu:

$$|w^1 \cdot y^*| > \left| \sum_{j \in U} w_j^0 \cdot y_j \right| \quad (\text{A.13})$$

trong đó U là tập các template có tương quan không mong muốn.

A.4 Phân tích tiệm cận

A.4.1 Lemma 6.1: Bậc của phương sai và kỳ vọng

Phân tích tiệm cận (*asymptotic orders*) cho phép xác định cách ensemble complexity tăng theo kích thước bài toán.

Lemma 6.1: Trong chế độ NTK với dữ liệu block-partitioned, với k' là kích thước đầu vào (số thành phần khác không):

$$\sigma^2(\hat{\mathbf{x}}) \in \mathcal{O}\left(\frac{1}{k'}\right) \quad (\text{A.14})$$

$$\mu(\hat{\mathbf{x}}) \in \mathcal{O}\left(\frac{1}{k'}\right) \quad (\text{A.15})$$

Hệ quả về tỷ lệ:

$$\frac{\sigma^2(\hat{\mathbf{x}})}{\mu^2(\hat{\mathbf{x}})} \in \mathcal{O}(k') \quad (\text{A.16})$$

Phân tích theo số block khớp $n_{\hat{\mathbf{x}}}$:

Khi đầu vào $\hat{\mathbf{x}}$ khớp với $n_{\hat{\mathbf{x}}}$ template (mỗi template ở một block khác nhau):

- **Kỳ vọng:** $\mu \propto n_{\hat{\mathbf{x}}} \cdot w^1$, tăng tuyến tính theo số block khớp
- **Phương sai:** $\sigma^2 \propto n_{\hat{\mathbf{x}}} \cdot (w^1)^2$, cũng tăng tuyến tính

- **Tỷ lệ:** $\sigma^2/\mu^2 \propto 1/n_{\hat{x}}$, giảm khi số block tăng

Kết quả này giải thích tại sao các bài toán với đầu vào sparse hơn (ít block khớp hơn) yêu cầu ensemble lớn hơn.

Phụ lục B

Mã nguồn cài đặt mô phỏng

Phụ lục này mô tả cấu trúc mã nguồn và quy trình thực thi mô phỏng để xác minh các kết quả lý thuyết.

B.1 Môi trường và thư viện

B.1.1 Thư viện Neural Tangents

Nghiên cứu sử dụng thư viện *Neural Tangents* (Google Research) để tính toán NTK và dự báo đầu ra của mạng nơ-ron trong giới hạn chiều rộng vô hạn.

Đặc điểm chính:

- Tính toán closed-form của kernel NTK cho các kiến trúc mạng phổ biến
- Hỗ trợ *NTK predictor* để dự báo đầu ra mà không cần huấn luyện thực tế
- Tích hợp với hệ sinh thái JAX để tối ưu hóa hiệu năng

B.1.2 Nền tảng JAX

JAX (Google) được sử dụng làm backend cho các tính toán ma trận hiệu năng cao:

- Hỗ trợ tự động vi phân (automatic differentiation) cho tính toán gradient
- Tối ưu hóa JIT compilation cho tốc độ thực thi
- Hỗ trợ GPU/TPU để xử lý ma trận lớn

Cài đặt môi trường:

```
1 pip install jax jaxlib neural-tangents
```

B.2 Cấu trúc mã nguồn

Toàn bộ mã nguồn được công khai tại GitHub:

https://github.com/BuhDuy256/add_multiply_execute_algo_instructions_with_neural_networks

Repository được tổ chức thành các module chính:

B.2.1 Modules định nghĩa Template

Mỗi thuật toán có module riêng định nghĩa các mẫu (templates):

- `templates/addition.py`: Định nghĩa 8 template cho mỗi vị trí bit của bộ cộng ripple-carry
- `templates/multiplication.py`: Template cho thuật toán shift-and-add
- `templates/sbn.py`: Template cho chỉ thị Subtract and Branch if Negative

B.2.2 Scripts mã hóa dữ liệu

Module `encoding/` chịu trách nhiệm mã hóa trực chuẩn (*orthonormal encoding*):

- Chuyển đổi template thành vector block-partitioned
- Đảm bảo tính trực giao giữa các block
- Chuẩn hóa vector để có norm phù hợp

B.2.3 Scripts tính toán Ensemble

Module `ensemble/` thực hiện:

- Tính toán ensemble mean từ nhiều mô hình
- Áp dụng hàm rounding để chuyển đầu ra về $\{-1, +1\}$
- Đánh giá độ chính xác trên tập kiểm tra

B.3 Quy trình thực thi mô phỏng

Quy trình mô phỏng gồm bốn bước chính:

B.3.1 Bước 1: Khởi tạo cấu trúc Block

Với thuật toán hoạt động trên l bit, khởi tạo cấu trúc block:

```
1 def initialize_blocks(l, algorithm_type):
2     """
3     Khởi tạo cấu trúc block cho thuật toán
4     Args:
5         l: Số bit đầu vào
6         algorithm_type: 'addition', 'multiplication', 'sbn'
7     Returns:
8         block_config: Cấu hình block cho từng vị trí
9     """
10    if algorithm_type == 'addition':
11        n_blocks = 1 # Mọi vị trí bit một block
12        block_sizes = [3] * l # 3 bit đầu vào mỗi block
13    # ...
14    return block_config
```

B.3.2 Bước 2: Tạo tập dữ liệu huấn luyện

Sinh tất cả các template từ cấu hình block:

```
1 def generate_training_set(block_config):
2     """
3     Tao tap du lieu huan luyen tu cac template
4     Returns:
5         X: Ma tran dau vao (M x d)
6         Y: Ma tran nhan (M x output_dim)
7     """
8     templates = []
9     for block_id, config in enumerate(block_config):
10         block_templates = enumerate_block_templates(config)
11         encoded = encode_block_partitioned(block_templates,
12                                           block_id)
13         templates.extend(encoded)
14     return np.array(templates)
```

B.3.3 Bước 3: Tính toán ma trận NTK

Sử dụng Neural Tangents để tính kernel:

```
1 from neural_tangents import stax
2
3 # Định nghĩa kiến trúc mạng
4 init_fn, apply_fn, kernel_fn = stax.serial(
5     stax.Dense(n_hidden),
6     stax.Relu(),
7     stax.Dense(output_dim)
8 )
9
10 # Tính NTK trên tập huấn luyện
11 K_train = kernel_fn(X_train, X_train, 'ntk')
12 K_test_train = kernel_fn(X_test, X_train, 'ntk')
13
14 # Tính NTK predictor
15 predictions = K_test_train @ np.linalg.solve(K_train, Y_train)
```

B.3.4 Bước 4: Thực thi lặp và làm tròn

Với các thuật toán đa bước (multi-step), thực thi lặp:

```
1 def iterative_execution(x_init, n_steps, predictor):
2     """
3     Thực thi lặp thuật toán
4     Args:
5         x_init: Trạng thái ban đầu
6         n_steps: Số bước lặp
7         predictor: Bộ dự báo NTK
8     Returns:
```

```

9         x_final: Trang thai ket thuc
10     """
11     x = x_init
12     for step in range(n_steps):
13         # Du bao buoc tiep theo
14         x_pred = predictor(x)
15         # Lam tron ve {-1, +1}
16         x = np.sign(x_pred)
17     return x

```

B.4 Kiểm chứng thực nghiệm

B.4.1 Trường hợp kiểm tra

Nhóm nghiên cứu đã xác minh tính đúng đắn trên các trường hợp cụ thể:

Phép cộng 10-bit:

- Kiểm tra tất cả 2^{20} cặp đầu vào (hai số 10-bit)
- Độ chính xác đạt 100% sau khi làm tròn
- Thời gian tính toán: dưới 5 phút trên GPU

Phép nhân 8-bit:

- Kiểm tra ngẫu nhiên 10,000 cặp đầu vào
- Độ chính xác đạt 100% cho tất cả các trường hợp
- Xác nhận tính đúng đắn của cơ chế thực thi lặp

B.4.2 Kết quả hội tụ

Các thực nghiệm ghi nhận sự hội tụ tuyệt đối (exact convergence):

- Mọi bit đầu ra đều có dấu đúng sau làm tròn
- Không có accumulated error qua các bước lặp
- Kết quả khớp hoàn toàn với tính toán số học tiêu chuẩn

Kết quả này xác nhận tính đúng đắn của cả khung lý thuyết lẫn triển khai mã nguồn.